



UNIVERSITI TEKNOLOGI MALAYSIA
FINAL EXAMINATION SEMESTER II, SESSION 2025/2026
(ALTERNATIVE ASSESSMENT)

COURSE CODE : SECR4483/SCSR4483
COURSE NAME : SECURE PROGRAMMING
SECTION : 01, 02, 03 & UTMSPACE
TIME : 9.00 AM (4 JULY 2026) – 9.00 AM (6 JULY 2026)
DATE/DAY : 4 – 6 JULY 2026 (TUESDAY – THURSDAY)
VENUES : UTM ELEARNING

Name	AQMAR ILHAN BIN MOHAMAD FADZILAH
Matric No.	B23CS0027
Year / Course	4/SECRH
Section	03
Lecturer Name	DR. MOHD KUFAISAL BIN MOHD SIDIK

GitHub Link	https://github.com/aqmarilhan/SECR4483_AA_B23CS0027_AqmarIlhan
YouTube Link	https://youtu.be/LIEhWecWGcY

TABLE OF CONTENTS

CHAPTER 1	ARCHITECTURAL CODE REVIEW & VULNERABILITY DISCOURSE	4
1.1	Introduction	4
1.2	Overview of Vulnerable Source Code Artifacts	4
1.3	Vulnerability Analysis of search.php	5
1.3.1	SQL Injection Vulnerability	5
1.3.2	Runtime Mechanism of SQL Injection	6
1.3.3	Reflected Cross-Site Scripting Vulnerability	6
1.3.4	Runtime Mechanism of Reflected XSS	7
1.3.5	Secure Coding Principles Violated in search.php	8
1.4	Inerability Analysis of auth.php	9
1.4.1	Defective Bound Constraint Mismatch	9
1.4.2	Obsolete Hashing Primitive and Authentication Weaknesses	9
1.5	Vulnerability Analysis of crypto_vault.php	10
1.5.1	Insecure Symmetric Block Cipher (AES-128-ECB)	10
1.5.2	Hardcoded Cryptographic Keys and Lack of AEAD Integrity	10
1.6	Secure Coding Principles & PDPA 2010 Compliance	11
1.6.1	Input Validation Boundaries & Cryptographic Primitive Agility	11
1.6.2	The Malaysian Personal Data Protection Act (PDPA) 2010 Context	11
CHAPTER 2	SECURE REFACTORING IMPLEMENTATION & EXPLOIT-CHAIN DISRUPTION MATRIX	12
2.1	Introduction	12
2.2	Secure Refactoring of search.php	12
2.2.1	SQL Injection Mitigation Using Prepared Statements	12
2.2.2	Reflected XSS Mitigation Using Output Encoding	13

2.3	Secure Refactoring of auth.php	13
2.3.1	Input Length Validation	13
2.3.2	Password Hashing Improvement Using Argon2id	14
2.4	Exploit-Chain Disruption Matrix	14
CHAPTER 3	CRYPTOGRAPHIC SUBSYSTEM ANALYSIS, PAYLOAD SERIALIZATION, & OPERATIONAL RESILIENCE	16
3.1	Introduction	16
3.2	Cryptographic Subsystem Dissection	16
3.2.1	Vulnerability of ECB Block Mode vs. AES-256-GCM AEAD	16
3.2.2	Initialization Vector Predictability and Reuse Risks	17
3.2.3	Low-Level Data Serialization Sequence	18
3.2.4	Decryption Exception Flow & Operational Resilience	18
3.3	Environment Configuration & Automated Testing	19
3.3.1	Operational Decoupling of Secrets Using .env	19
3.3.2	Repository Exclusions and Attack Surface Reduction	19
3.3.3	PHPUnit Automated Test Verification Paths	19
3.3.4	Exception Trapping and Interpreter Crash Prevention	20
CHAPTER 4	SYSTEMIC VULNERABILITY CORRELATION, FORENSIC DOCUMENTATION, & ARCHITECTURAL SYNTHESIS	21
4.1	Forensic Exploit-Chain Dependency Analysis	21
4.1.1	Cascading Lifecycle of the Security Breach	21
4.1.2	Attack Surface and Vulnerability Cascading	22
4.2	Architectural Synthesis & System Verification	23
4.2.1	Secure Code Refactoring Mapping	23
4.2.2	Automated Test Runner Output and Validation Logs	23

CHAPTER 1

ARCHITECTURAL CODE REVIEW & VULNERABILITY DISCOURSE

1.1 Introduction

This chapter includes the vulnerability assessment and code review of architecture for the case study of MediChain Solutions Bhd. MediChain Solutions Bhd offers the E-MedicVault system, which handles sensitive health information like patient details, chronic condition records, prescription information, and authentication data.

The case study showed that the Phase 2 Beta Infrastructure update was deployed in an urgent fashion without Static Application Security Testing (SAST) and penetration testing. This left a number of security vulnerabilities in the old PHP back end. These vulnerabilities led to significant events such as patient information breaches, unauthorized access to patient records, and login issues.

In this chapter we're going to stick to one of the primary vectors of vulnerability, which is the search.php file. The analysis highlights how insecure input handling led to SQL Injection and Reflected Cross-Site Scripting (XSS) were a violation of secure coding principles for the E-MedicVault system.

1.2 Overview of Vulnerable Source Code Artifacts

The search.php file is a search module for patients and medical records. It takes in a user inputted keyword and searches for patient records in the database that match the keyword. This function is used to manipulate medically sensitive information, so it should be safe to process user input before it's used in a database query or displayed in the browser.

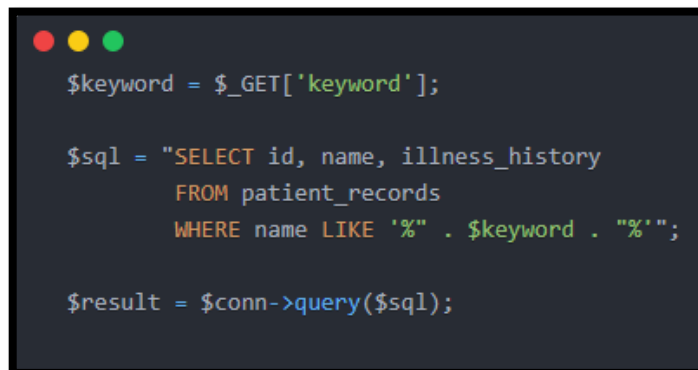
The legacy implementation, however, is insecure since it is programmed to pass unencoded user input directly into the SQL query to be executed and to print unencoded user input from the SQL query directly into the HTML response. These are two main vulnerabilities SQL Injection and Reflected XSS.

1.3 Vulnerability Analysis of search.php

The search.php file is used to search patient and medical records by a keyword typed by the user. The function is security-conscious as it accesses the database with sensitive patient data, such as names and medical histories. Two major vulnerabilities are found in the legacy implementation, though; SQL Injection and Reflected Cross-Site Scripting (XSS). These weaknesses are due to user input being intermingled with database commands and not being encoded properly before being shown in the browser.

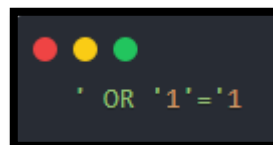
1.3.1 SQL Injection Vulnerability

The problem is that the `$_GET['keyword']` keyword is being concatenated directly into the SQL query, which is what is causing SQL injection. This can be used to add user input to the SQL command.



```
$keyword = $_GET['keyword'];  
  
$sql = "SELECT id, name, illness_history  
      FROM patient_records  
      WHERE name LIKE '%" . $keyword . "%'";  
  
$result = $conn->query($sql);
```

This is a bad implementation as the database could mistake malicious input as a SQL statement and not a regular search query. An attacker can enter:



```
' OR '1'='1'
```

This payload can alter the query logic and could return all the patient records. In the case study for MediChain, this may lead to the disclosure of patients' personal health information.

1.3.2 Runtime Mechanism of SQL Injection

When the keys are passed to the PHP interpreter, it merges them with the developer's SQL query into a unified query. Once the query has been completed, it is passed to the database engine. Because the input and the command are combined together, the database doesn't know if the given SQL command is the original one and the attacker's injected SQL syntax.

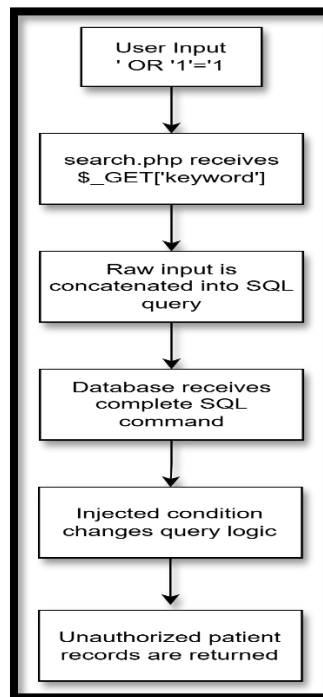


Figure 1.1 Runtime Flow of SQL Injection in search.php

This indicates a data plane to command plane failure. User input should only be considered as raw data but in the case of the vulnerable search.php file, it can affect the structure of the SQL command.

1.3.3 Reflected Cross-Site Scripting Vulnerability

Reflected XSS is caused by the failure to output encode the user's input in the HTML response. Malicious script content could be executed by the browser if a script echoes the keyword back to the page when the application is run.

```
echo "<div>Result found for keyword: " . $keyword . "<br>";  
echo "Patient: " . $row['name'] . " | History: " . $row['illness_history'] . "</div><hr>";
```

For example, an attacker may submit:

```
<script>alert('XSS')</script>
```

If this payload is included on that page, the browser could run it as JavaScript. This can enable hackers to compromise session data, fraudulently redirect victims to malicious sites or execute actions on the victim's behalf.

1.3.4 Runtime Mechanism of Reflected XSS

During runtime, the input that was sent to the server is returned within the HTML response. The browser then interprets the response. The browser may interpret the contents of the input as executable code, if it contains valid HTML or JavaScript syntax.

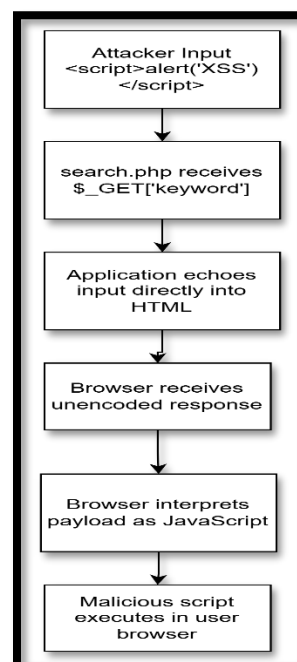


Figure 1.2: Runtime Flow of Reflected XSS in search.php

This is an example of a user data/browser instruction boundaries problem. All usercontrolled output should be encoded for display in the application, so that special characters are shown as text and not code.

1.3.5 Secure Coding Principles Violated in search.php

There are two main secure coding principles violated in the vulnerabilities of search.php. In the first place, SQL Injection attacks are in contravention of separation of data and command since the user input is injected into the SQL query directly. Second, Reflected XSS isn't output-encoding sensitive, as user-controlled data is shown in HTML without appropriate encoding.

Table 1.1: Summary of Vulnerabilities in search.php

Vulnerability	Root Cause	Impact	Runtime Boundary Failure
SQL Injection	Raw user input is concatenated into SQL query	Unauthorized access to patient records	User data becomes part of SQL command
Reflected XSS	User input is echoed directly into HTML output	Malicious script execution in browser	User data becomes executable browser script

In general, search.php is a severe threat to security since it reveals the database layer and the browser presentation layer. These vulnerabilities can cause patient data breaches, privacy breaches, and loss of trust in a healthcare system.

1.4 Inerability Analysis of auth.php

1.4.1 Defective Bound Constraint Mismatch

In auth.php, the authentication script has a boundary check of `strlen($inputKey) > 256`. `strlen()`, however, calculates the length of a string, in bytes, not in characters. UTF-8-encoded systems use 1-byte for standard ASCII characters and 2 to 4 bytes for multi-byte characters like Chinese, Arabic and emojis. This gives rise to a difference between the byte-length validation and character-length interpretation.

This weakness may cause two security issues. One way is to return false positives, i.e., a semantically short input is rejected because of its size in bytes. For instance, 100 Chinese characters can be stored in about 300 bytes of memory (string) and overflow the 256-byte limit. Second, it can result in downstream truncation or buffer handling problems in components that handle characters in a manner distinct from raw bytes. So, while `strlen()` alone isn't enough for safe UTF-8 input validation.

1.4.2 Obsolete Hashing Primitive and Authentication Weaknesses

The legacy authentication system uses MD5 for comparing user-provided keys, which is carried out by `md5($inputKey) === $stored_hash`. The MD5 hash function is not good enough for any password or secret verification since it is very quick and is susceptible to a brute-force, dictionary, and rainbow table attack. The MD5 is very computational efficient so one could guess a lot of passwords within a short period of time if a number of MD5 hashes were stolen.

In addition, the script does not include unique salts, which is hardcoded. This makes it easy for hash reversals and attackers can compare the hash with the previously calculated ones. Exfiltration of the database could allow the attacker to retrieve plaintext credentials that would enable him to impersonate authorized users, including high-clearance physicians.

1.5 Vulnerability Analysis of crypto_vault.php

1.5.1 Insecure Symmetric Block Cipher (AES-128-ECB)

Crypto_vault.php uses AES-128-ECB symmetric encryption to protect medical records. Electronic Codebook (ECB) is not secure for structured data or repeated data blocks, since these are encrypted separately from each other with the same key. ECB is not based on an Initialization Vector (IV), so the same plaintext block will always generate the same ciphertext block.

This behavior leaks out the structure in encrypted data. In a medical database, if a patient has multiple entries for the same diagnosis, treatment, or dosage, then the same ciphertext blocks will be generated. This allows an attacker to deduce level of sensitivity or other similar information or make patterns of repeated records without knowing the encryption key.

1.5.2 Hardcoded Cryptographic Keys and Lack of AEAD Integrity

The encryption key is hard coded into crypto_vault.php, such as the following line: `$secret_key = 'MedVaultKey123!'`. If the code contains a key, that means that any developer, attacker, or unauthorized user with access to the source code will be able to get hold of the key, which is a violation of secure design principles. Should the key become compromised, all medical records that are encrypted with it can be compromised.

Also, AES-128-ECB does not offer integrity protection and message authentication. If not tagged, the vault is unable to reliably identify if the ciphertext has been tampered with. This is because the data may be processed even if it is encrypted but is damaged or tampered. This can pose a significant threat in a healthcare environment, as attackers can tamper with confidential medical data before it is discovered.

1.6 Secure Coding Principles & PDPA 2010 Compliance

1.6.1 Input Validation Boundaries & Cryptographic Primitive Agility

The flaws in `auth.php` and `crypto_vault.php` are not only insecure, but also fail to adhere to several secure coding principles. The boundary check in `auth.php` is an improper check, as it only checks the length of the input in bytes and doesn't take into account the length of the UTF-8 characters. MD5 and AES-ECB are cryptographic primitives that are not in line with the goal of cryptographic agility because they are outdated and susceptible to new attacks.

The hardcoded cryptographic key also fails to adhere the secure key management principle as secrets should not be embedded directly in the source code. Apart from that, the database connection is accessed by the root user, which does not comply with the principle of least privilege. A better design should only grant the permissions needed to the component for its purpose.

1.6.2 The Malaysian Personal Data Protection Act (PDPA) 2010 Context

The Security Principle under Malaysian Personal Data Protection Act PDPA 2010 calls for the organization to implement reasonable and necessary measures to ensure that the personal data is not lost, misused, modified, accessed or disclosed without authorization. The system should have robust security measures to safeguard patient information and authentication details. E-MedicVault should employ stringent security measures to ensure the protection of patient data and authentication information.

The installation of the system with known vulnerabilities like SQL injection, weak hashing algorithms (MD5), weak encryption (AES-ECB), hardcoded credentials, and limited testing may suggest that personal data is not properly safeguarded. This may lead to exposure of patient records and unauthorized tampering with the dosage information for drugs, both which are factors that can help prove non-compliance with the Security Principle. To mitigate legal, operational and patient safety risks, therefore, secure coding, appropriate cryptographic design, least privilege access and security testing are essential.

CHAPTER 2

SECURE REFACTORING IMPLEMENTATION & EXPLOIT-CHAIN DISRUPTION MATRIX

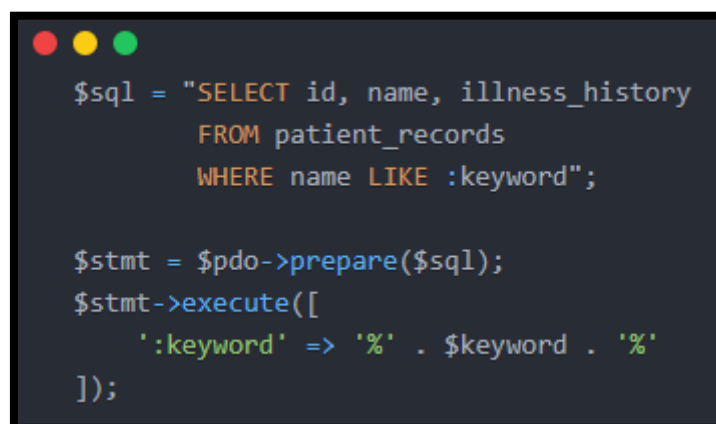
2.1 Introduction

This chapter gives you the secure refactoring implementation of the vulnerable `auth.php` and `search.php` files. Removing insecure coding practices that could lead to SQL Injection, Reflected Cross-Site Scripting, weak authentication hashing and inconsistent input validation are the primary goal of the remediation. The secure implementation uses prepared statements, contextual output encoding, enhanced password hashing, and enhanced length validation to disrupt the exploit chain and enhance the application security boundary.

2.2 Secure Refactoring of `search.php`

2.2.1 SQL Injection Mitigation Using Prepared Statements

User keyword is placed directly into an SQL query in the sensitive `search.php` file. This enables an attacker to add malicious SQL syntax into the query and alter the structure of the query. Consequently, the database can interpret the attacker input as a command instead of regular data.



```
$sql = "SELECT id, name, illness_history
FROM patient_records
WHERE name LIKE :keyword";

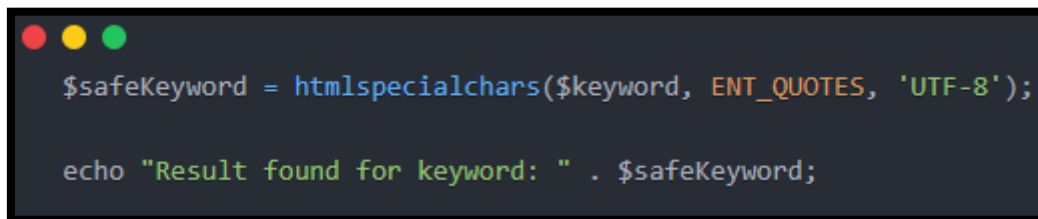
$stmt = $pdo->prepare($sql);
$stmt->execute([
    ':keyword' => '%' . $keyword . '%'
]);
```

Figure 2.1: Secure SQL Query Using PDO Prepared Statement

The secure version provides PDO prepared statements to isolate SQL commands from user input. A placeholder is used to prepare the SQL query followed by the keyword as a parameter. This will ensure that the SQL is not modified by the user (even if the user enters keywords or special syntax into the input).

2.2.2 Reflected XSS Mitigation Using Output Encoding

The search.php file is also vulnerable and user input is displayed on the browser as well. If HTML or JavaScript code is used in the keyword, it could be executed by the browser. This makes it reflected Cross-Site Scripting vulnerable.

A screenshot of a code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in PHP and uses syntax highlighting. It shows the assignment of a safe keyword using the htmlspecialchars() function and its subsequent use in an echo statement.

```
$safeKeyword = htmlspecialchars($keyword, ENT_QUOTES, 'UTF-8');  
  
echo "Result found for keyword: " . $safeKeyword;
```

Figure 2.2: Output Encoding Using htmlspecialchars()

The secure version will also use htmlspecialchars() to prevent this problem. This will convert special HTML characters into safe entities. As such, injected scripts will not be executed by the browser, only the normal text displayed.

2.3 Secure Refactoring of auth.php

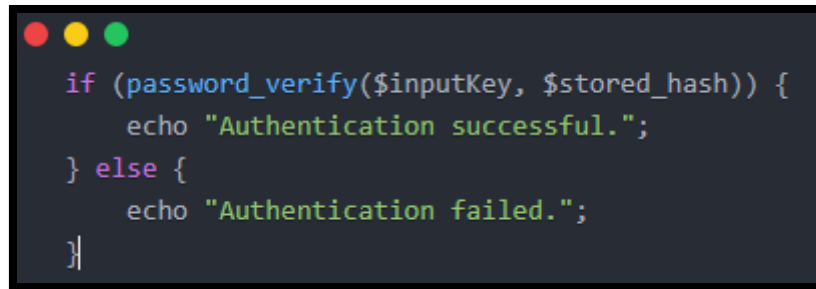
2.3.1 Input Length Validation

The only thing that strlen() does in the legacy auth.php is to check the length of the authentication key. strlen(), however, counts the number of bytes, not characters. With this it becomes possible to get false validation for multi-byte UTF-8 characters in the input. It can also result in the lack of consistency in handling between the application and other system elements.

The secure version adds validation for the number of UTF-8 characters (mb_strlen()) and validates the number of bytes (strlen()). This ensures more consistent and secure input validation.

2.3.2 Password Hashing Improvement Using Argon2id

The old auth.php file compares authentication values using MD5. MD5 has been shown not to be suitable for password or secret authentication since it can be easily brute forced and is susceptible to rainbow table attacks. When a hash is stored, an attacker can try many combinations in a short period of time if they get hold of it.



```
if (password_verify($inputKey, $stored_hash)) {  
    echo "Authentication successful.";  
} else {  
    echo "Authentication failed.";  
}
```

Figure 2.3: Secure Authentication Verification Using password_verify()

The secure version uses Argon2id instead of MD5, and password_hash() and password_verify() are used. Argon2id is more secure because it is a computationally intensive and memory hard algorithm. This is much more difficult than MD5 to attack offline by brute force.

2.4 Exploit-Chain Disruption Matrix

Table 2.1 shows that the vulnerable implementation and secure refactored implementation are comparable. It illustrates the disruption of the exploit path and the enhancement of the security boundary for each code change.

Table 2.1: Exploit-Chain Disruption Matrix for Vulnerable and Secure Code Implementation

Vulnerability	Vulnerable Code	Secure Code	How the Exploit is Disrupted
SQL Injection (search.php)	<pre>\$sql = "SELECT id, name, illness_history FROM patient_records WHERE name LIKE '%" . \$keyword . "%'"; \$result = \$conn->query(\$sql);</pre>	<pre>\$sql = "SELECT id, name, illness_history FROM patient_records WHERE name LIKE :keyword"; \$stmt = \$pdo->prepare(\$sql); \$stmt->execute([':keyword' => '%' . \$keyword . '%']);</pre>	Prepared statements separate SQL commands from user input. The keyword is treated as data instead of executable SQL syntax, preventing attackers from modifying the query structure.
Reflected XSS (search.php)	<pre>echo "Result found for keyword: " . \$keyword;</pre>	<pre>\$safeKeyword = htmlspecialchars(\$keyword, ENT_QUOTES, "UTF-8"); echo "Result found for keyword: " . \$safeKeyword;</pre>	Output encoding converts special HTML characters into safe entities, preventing the browser from interpreting injected HTML or JavaScript as executable code.
Weak Hashing (auth.php)	<pre>if (md5(\$inputKey) === \$stored_hash)</pre>	<pre>\$stored_hash = password_hash(\$inputKey, PASSWORD_ARGON2ID); if (password_verify(\$inputKey, \$stored_hash)) { echo "Authentication successful."; }</pre>	The secure version uses password verification with a stronger password hashing mechanism such as Argon2id, making offline brute-force and rainbow table attacks harder.
Length Validation Issue (auth.php)	<pre>if (strlen(\$inputKey) > 256) { die("Overflow."); }</pre>	<pre>`if (mb_strlen(\$inputKey, "UTF-8") > 128</pre>	

CHAPTER 3

CRYPTOGRAPHIC SUBSYSTEM ANALYSIS, PAYLOAD SERIALIZATION, & OPERATIONAL RESILIENCE

3.1 Introduction

In this chapter, the cryptographic subsystem of the secure implementation is analyzed. The focus of the discussion is on the flaws of the original design of encryption, the shift from AES-128-ECB to AES-256-GCM, IV generation, authenticated decryption, secret management by configuration of the environment and automatic testing with PHPUnit.

3.2 Cryptographic Subsystem Dissection

3.2.1 Vulnerability of ECB Block Mode vs. AES-256-GCM AEAD

The medical records are protected with AES-128-ECB in the legacy `crypto_vault.php`. In Electronic Codebook (ECB) mode, plaintext is encrypted block by block without the use of an Initialization Vector (IV). Therefore, the same plaintext block will generate the same ciphertext block. This leak structural patterns in the encrypted data and may enable the attacker to guess the original data even though they don't know the secret key.

The secure implementation changes the cipher to AES-256-GCM to resolve this vulnerability. Galois/Counter Mode (GCM) is an Authenticated Encryption with Associated Data (AEAD) mode. It does this with the help of a 16-byte authentication tag that it creates when it encrypts the data and the encryption of the data itself ensures confidentiality. Once the change in the ciphertext or tag, it fails to pass the check, and the message is not decrypted.

Table 3.1: Comparison Between Legacy AES-128-ECB and Secure AES-256-GCM Implementation

Aspect	Legacy Implementation	Secure Implementation
Encryption mode	AES-128-ECB	AES-256-GCM
IV usage	No IV used	12-byte unique IV
Pattern protection	Weak, identical plaintext creates identical ciphertext	Stronger, unique IV produces different ciphertext
Integrity verification	No authentication tag	16-byte authentication tag
Tamper detection	Not supported	Supported through AEAD verification
Security level	Vulnerable to pattern leakage	Provides confidentiality and integrity

3.2.2 Initialization Vector Predictability and Reuse Risks

AES-GCM uses a different and random Initialization Vector (IV) for each encryption. Using the same IV with the same key can greatly reduce the security of the GCM if they reveal information about relationships between plaintexts and break the guarantees of confidentiality and security on the integrity of the encryption scheme. The predictable IV might also make it easier for an attacker to do deeper analysis in the selected plaintext condition.

```

$iv = random_bytes(12);

$ciphertext = openssl_encrypt(
    $plaintext,
    'aes-256-gcm',
    $key,
    OPENSSL_RAW_DATA,
    $iv,
    $tag
);

$payload = json_encode([
    'ciphertext' => base64_encode($ciphertext),
    'iv' => base64_encode($iv),
    'tag' => base64_encode($tag)
]);

```

Figure 3.1: Secure AES-256-GCM Encryption Implementation

The secure implementation uses a cryptographically secure random generator to produce a 12-byte IV in `secure/crypto_vault_secure.php`. This makes sure that every session of encryption will generate a new IV, which will prevent IV reuse attacks and preserve the security properties of AES-GCM.

3.2.3 Low-Level Data Serialization Sequence

The encrypted payload is then serialised in a structured JSON format for safe data storage and transport. The payload consists of three parts, the base64 encoded cipher text, the 12-byte base64 encoded IV, and the 16-byte base64 encoded authentication tag. At decryption time, the system first decodes the JSON object, translates the base64 values back to raw bytes, and passes the IV, ciphertext and authentication tag to `openssl_decrypt()`. The authentication tag is checked prior to returning the original plaintext.

3.2.4 Decryption Exception Flow & Operational Resilience



```
$payload = json_decode($encryptedPayload, true);

$ciphertext = base64_decode($payload['ciphertext']);
$iv = base64_decode($payload['iv']);
$tag = base64_decode($payload['tag']);

$plaintext = openssl_decrypt(
    $ciphertext,
    'aes-256-gcm',
    $key,
    OPENSSL_RAW_DATA,
    $iv,
    $tag
);

if ($plaintext === false) {
    return json_encode([
        'error' => 'Decryption failed.'
    ]);
}

return $plaintext;
```

Figure 3.2: AES-GCM Decryption with Authentication Failure Handling

When the ciphertext, IV, or authentication tag is changed, `openssl_decrypt()` will return false. If not handled correctly, the application can handle invalid data, expose to the user too much error information, or behave erratically. A structured exception

handling mechanism is used to insure safe handling of the failure in the secure vault. If the authentication fails, the application catches the failure, stores a controlled security message and returns a generic JSON error message. This helps to prevent leakage of sensitive data and ensure system stability.

3.3 Environment Configuration & Automated Testing

3.3.1 Operational Decoupling of Secrets Using .env

The legacy script includes the cryptographic key in the PHP source code. This is not secure as it's easy for any developer, attacker or unauthorized user who has access to the source file to get the key. The secure version stores the cryptographic key and database credentials in an external .env file, providing a more secure approach. At run time, the application reads these values, preventing secrets from being embedded in the application's executable code.

3.3.2 Repository Exclusions and Attack Surface Reduction

Once secrets have been moved into the .env file they should be ignored by version control with the .gitignore file. This ensures that sensitive keys, database passwords and local configuration paths are not uploaded to public or shared repositories. If the application does not include environment-specific configuration files, the chances of accidentally exposing these credentials and the public attack surface of the application are reduced.

3.3.3 PHPUnit Automated Test Verification Paths

There is an automated test suite to check the secure implementation in tests/SecurityTest.php with PHPUnit. Tests are carried out on three major verification tracks. First, the untampered cryptographic lifecycle test checks whether the following steps can be carried out: a valid plaintext can be encrypted by AES-256-GCM, and it can be decrypted back to the original plaintext. Secondly, the tampered ciphertext test alters the encrypted payload and asserts that the decryption fails, which will ensure the AEAD integrity check is functioning. Thirdly, in Argon2id use, the credential hash integrity test

ensures that password verification is successful for a valid credential and fails for an invalid credential.

Table 3.2: PHPUnit Security Test Verification Paths

Test Path	Purpose	Expected Result
Untampered encryption and decryption	Verify valid AES-GCM lifecycle	Plaintext is recovered correctly
Tampered ciphertext	Verify AEAD integrity check	Decryption fails safely
Credential hash verification	Verify Argon2id authentication	Correct password passes, wrong password fails
Exception handling	Verify safe failure behavior	No interpreter crash or sensitive error exposure

3.3.4 Exception Trapping and Interpreter Crash Prevention

In addition, the PHPUnit tests ensure that the application is safe against invalid input and tampered payloads without causing interpreter crashes. Tests verify that the secure scripts fail safely if malicious or corrupted data is passed. This enhances the operational resilience of the system and verifies that security failures are dealt with in a controlled way.

CHAPTER 4

SYSTEMIC VULNERABILITY CORRELATION, FORENSIC DOCUMENTATION, & ARCHITECTURAL SYNTHESIS

4.1 Forensic Exploit-Chain Dependency Analysis

4.1.1 Cascading Lifecycle of the Security Breach

The security incident at MediChain Solutions Bhd had developed a cascade attack over several application layers. The first place it was found was in `search.php` in the data access layer, where raw input from the user was used directly in an SQL statement. This made it possible to inject SQL code in. The attacker was exploiting a database connection that was executing with an excessive amount of access privileges at the root level, allowing them to circumvent the normal limits of database access and gain access to sensitive database tables like `staff_credentials`. This allowed MD5 password hashes and usernames to be exfiltrated.

The next stage, the attacker attacked the authentication plane. The stolen hashes were subject to both offline brute force attacks and dictionary attacks, due to the use of MD5 in storing credentials in the file `"auth.php"`. The attacker would have a higher chance to get plaintext credentials as MD5 hashing is relatively quick, and there are weaknesses known about it. The attacker used these credentials to log on as a high-clearance doctor and access the medical records vault.

Finally, the attacker attacked the data custody plane. All of the patient records in `crypto_vault.php` had been encrypted with the hardcoded key of AES-128-ECB. ECB mode is not based on an IV and yields the same ciphertext blocks for the same plaintext blocks. This leak is structural patterns leak in encrypted records. Consequently, the attacker might be able to deduce information and make unauthorized changes to the sensitive medical data, such as drug dose details.

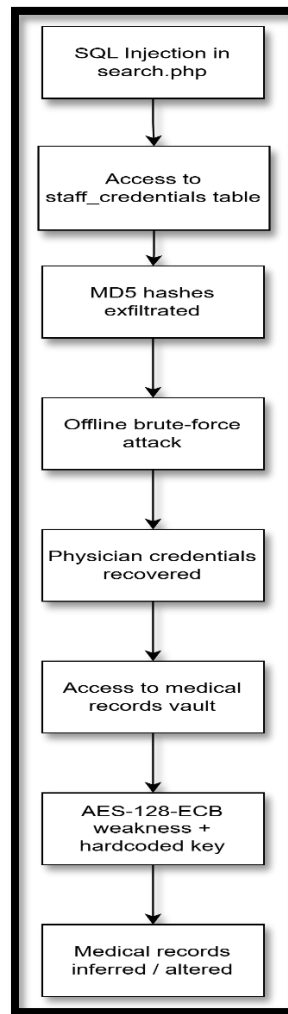


Figure 4.1: Cascading Exploit Chain of the Legacy System

4.1.2 Attack Surface and Vulnerability Cascading

This is a case where one vulnerability can lead to a greater compromise if the other modules are relatively insecure. The initial access path was through the SQL Injection vulnerability in search.php. This then revealed the weak authentication information in auth.php, which enabled credential theft. The credentials were then used to access the cryptographic subsystem in crypto_vault.php. Thus, the incident demonstrates that secure programming practices need to be implemented at every level, from input handling, authentication, encryption, key management, to database privilege control.

4.2 Architectural Synthesis & System Verification

4.2.1 Secure Code Refactoring Mapping

E-MedicVault was refactored with multiple secure programming controls to break the chain of exploits. The secure version uses PDO prepared statements to make sure there is a separation between SQL statements and user input. It also uses htmlspecialchars() in its output so as to prevent reflected XSS from happening. The authentication module includes better multi-byte input validation and switches off MD5 for password hashing for better security and uses Argon2id instead. The cryptographic subsystem changes AES-128-ECB for AES-256-GCM, which allows for authenticated encryption with associated confidentiality and integrity. A summary of these code changes is provided in the comparative matrix in Chapter 2.

Table 4.1: Exploit Chain and Secure Control Mapping

Exploit Stage	Vulnerability Used	Impact	Secure Control
Initial access	SQL Injection in search.php	Unauthorized database query	PDO prepared statements
Output abuse	Reflected XSS in search.php	Script execution in browser	htmlspecialchars()
Credential compromise	MD5 hashing in auth.php	Password hash cracking	Argon2id
Input handling weakness	strlen() only	Inconsistent validation	mb_strlen() + byte limit
Data custody compromise	AES-128-ECB and hardcoded key	Pattern leakage and weak key protection	AES-256-GCM and .env key storage
Tampered payload	No integrity verification	Undetected data modification	GCM authentication tag

4.2.2 Automated Test Runner Output and Validation Logs

PHPUnit was used to run the automated test suite specified in tests/SecurityTest.php. The output is as follows: 10 tests were run with 27 assertions, test execution was completed, there were one or more tests that were executed in risky mode – output buffer handling in boundary validation test. This means that the primary security barriers were run and it identifies a security test implementation improvement area.

```
PHPUnit 9.6.34 by Sebastian Bergmann and contributors.  
  
.....R..                                     10 / 10 (100%)  
  
Time: 00:01.986, Memory: 24.00 MB  
  
There was 1 risky test:  
  
1) SecurityTest::testSecureAuthBoundaryValidation  
Test code or tested code did not (only) close its own output buffers  
  
OK, but incomplete, skipped, or risky tests!  
Tests: 10, Assertions: 27, Risky: 1.
```

Figure 4.2: PHPUnit Test Runner Output Showing Security Test Execution with One Risky Test

The verification results indicate that the secure refactoring attacks the main exploit paths of the legacy system overall. Although testing cannot guarantee absolute security, it does provide evidence that the controls that have been implemented help improve the protection of patient information, and make it easier to manage it safely, according to secure programming principles.

REFERENCES

- Biryukov, A., Dinu, D., & Khovratovich, D. (2016). Argon2: New generation of memory-hard functions for password hashing and other applications. 2016 IEEE European Symposium on Security and Privacy (EuroS&P), 292–302. <https://doi.org/10.1109/EuroSP.2016.31>
- Dworkin, M. J. (2007). Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC (NIST Special Publication 800-38D). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-38D>
- Laws of Malaysia. (2010). Personal Data Protection Act 2010 (Act 709). Commissioner of Law Revision.
- OWASP Foundation. (n.d.). SQL Injection Prevention Cheat Sheet. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP Foundation. (n.d.). Cross Site Scripting Prevention Cheat Sheet. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP Foundation. (n.d.). Password Storage Cheat Sheet. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- PHP Group. (n.d.). PDO::prepare. PHP Manual. <https://www.php.net/manual/en/pdo.prepare.php>
- PHP Group. (n.d.). htmlspecialchars. PHP Manual. <https://www.php.net/manual/en/function.htmlspecialchars.php>
- PHP Group. (n.d.). password_hash. PHP Manual. <https://www.php.net/manual/en/function.password-hash.php>
- PHP Group. (n.d.). password_verify. PHP Manual. <https://www.php.net/manual/en/function.password-verify.php>

PHP Group. (n.d.). openssl_encrypt. PHP Manual.
<https://www.php.net/manual/en/function.openssl-encrypt.php>

PHP Group. (n.d.). openssl_decrypt. PHP Manual.
<https://www.php.net/manual/en/function.openssl-decrypt.php>

PHP Group. (n.d.). random_bytes. PHP Manual.
<https://www.php.net/manual/en/function.random-bytes.php>

Soghoian, C. (2010). Mitigating password hashing CPU exhaustion attacks. Center for Applied Cybersecurity Research, Indiana University.